FORGE · STRUCTURED AUTONOMY

THE FORGE PLAYBOOK

Found, run, and trust an AI software company — inside walls it cannot cross.

FREE SAMPLE · MODULES 0–1

Written alongside the AI company it describes

forgeframework.dev

Free sample · Modules 0–1 · v1.0

Module 0 — Read This First

You bought (or are sampling) a playbook about handing real work to AI agents. You are probably feeling two things at once: this could change how I work, and this could wreck my repo. Both instincts are correct, and this book takes both seriously.

This module exists so nothing later surprises you — not the cost, not the time, not the first unattended run. Ten minutes here saves you a stalled lab at 11pm.

What you need

A Claude subscription. The labs run inside Claude Code, so your Claude plan is the fuel line. The honest breakdown:

- **Modules 1–5 are light.** You drive Claude Code by hand in short sessions — tripping guardrails, founding your company, running single tasks. A Pro plan handles these fine.
- **Module 6 runs agents in supervised bursts.** Still workable on Pro, but you may hit your usage cap if you do the whole module in one sitting.
- **Module 7 is the unattended capstone.** The lab is a daytime run — agents work while you're out of the building — and the challenge is the overnight version, where they work while you sleep. For that night you want headroom — a Max plan is the difference between waking up to a night's work and waking up to a run that stalled at 2am on your cap.

One thing to be clear about: hitting a cap mid-lab is **annoying, not dangerous**. The safety hooks you install in Module 1 are deterministic programs, not model behavior — they keep working on any plan. A stalled run means less got done, never that something unsafe got done. On Pro, do Modules 1–6 comfortably, then either upgrade for the capstone or run it as a shorter evening flight.

There is no per-token billing anywhere in this book. Forge runs on your flat Claude subscription; the framework itself is open source and free.

A GitHub account with the `gh` CLI authenticated. From Module 2 onward, GitHub is your safety net — every piece of agent work arrives as a pull request you can inspect, merge, or close. Before you start, run:

```
gh auth status
```

If that errors, `gh auth login` fixes it. Two GitHub facts worth knowing before you commit to a plan: branch protection rules — which Module 5 leans on — are free on **public** repos but require a paid plan on private ones, and GitHub Actions minutes are capped on private repos. Your sandbox is a throwaway with nothing sensitive in it, so make it public and both problems disappear.

macOS or Linux, and a terminal you are comfortable in. The framework was built and battle-tested on macOS. Linux works; where the book touches OS-specific machinery (Module 7's daemon autostart uses launchd on macOS), the installer offers a systemd path and the text flags the difference. Windows is not supported natively; WSL2 may work, but the labs were not written against it, and this book does not pretend otherwise.

A few standard tools. The installer checks its own prerequisites — Claude Code CLI, Python 3.10+, `uv`, and git are required; `ruff` and Node are optional. Module 1 walks you through getting the framework; once you have it locally, preflight your machine with:

```
./install.sh --check
```

It prints a pass/fail line per tool and tells you exactly how to install anything missing. No guessing.

Time: roughly 1–3 hours per module. The full table is below. You can spread a module across evenings — every lab ends at a stable state, and nothing rots if you stop for three days.

Time and intensity, module by module

"Agent intensity" is about your usage budget and your attention. **Low** means a handful of short exchanges you drive by hand. **Medium** means sustained sessions where Claude does multi-step work while you watch. **High** means agents running with you out of the loop — usage accumulates while you are not typing, and a Pro plan can run dry mid-flight.

Module	Focus	Hands-on time	Agent intensity
1	Structured autonomy — install, then break your own rules	1–2 h	Low
2	Found your company — hire your first AI employees	1–2 h	Low
3	First work by hand — one task, end to end	2–3 h	Medium

Module	Focus	Hands-on time	Agent intensity
4	Plan like a CEO — roadmaps and the admission gate	1–2 h	Medium
5	The GitHub flow safety net — PRs, labels, branch protection	2–3 h	Medium
6	Go autonomous with a canary	2–3 h	High
7	Operate: steer don't row — the unattended capstone	~1 h active, plus hours away	High
8	Scale and economics — dials, numbers, and choosing your execution provider	1–2 h	Low
9	Feed the company — the scout loop	1–2 h	Low

Two budget notes baked into the labs. First, the framework ships configured for 5 parallel workers; the labs pin it to 1–2 through Module 7 — kinder to your laptop and your usage cap, and raising it is precisely the decision Module 8 teaches you to make with data. Second, Modules 6 and 7 are the only places significant usage happens without you present. Everything before that, you can stop by closing the terminal.

The sandbox contract

Here is the deal this book makes with you, and it is not negotiable:

In Module 1 you create one throwaway project. That single sandbox carries through all nine modules. You never, at any point, run a lab against a repository you care about.

This matters for a reason beyond caution. The labs deliberately provoke failure — you will attempt a push that gets blocked, write a fake credential that gets refused, and in Module 6 send a booby-trapped task to see whether the admission gate catches it. Provoking failure is how you learn to trust the guardrails: a guardrail you have never seen say "no" is a guardrail you only believe in. That kind of deliberate arson belongs in a building you own and nobody lives in.

The sandbox also accumulates. The company you found in Module 2 works in it, the plans from Module 4 live in it, the overnight run in Module 7 fills its PR list. By Module 8 it is a small, real history of agent-produced work you can measure — and Module 9's closing audit still leans on it. Deleting it mid-book means redoing setup; the modules assume continuity.

When you finish the book and want Forge on your actual project, that is a separate, deliberate migration — the full Playbook covers how to introduce it to a real codebase safely, starting with the files you protect before anything else.

The GitHub bridge

Module 1 runs entirely local — no remote, on purpose, so the first guardrail you trip (a blocked push) can't touch anything even in principle.

At the **end of Module 2**, you connect the sandbox to GitHub: create the remote repository, make your first push, enable GitHub Actions, and create two labels the automation uses to route pull requests for human attention versus auto-merge. It takes about ten minutes.

I am telling you now so the shift does not feel like scope creep when it arrives. From Module 3 onward, everything your AI employees produce shows up as a pull request on that repo; from Module 5 onward, GitHub's own branch protection becomes a second, independent layer of your safety net — one that holds even if every local hook were deleted. The bridge is short, but everything after it stands on it.

How to read this book

Every module runs the same rhythm:

1. **Concept** — the idea, with the real numbers that motivated it.
2. **Walkthrough** — me doing the thing, annotated, so you see the expected shape before you try.
3. **Lab** — you doing the thing, in your sandbox, with exact commands.
4. **Challenge** — an optional harder version with deliberate gaps you must discover yourself. Skippable, but the challenges are where the understanding compounds.
5. **War story** — something that genuinely went wrong in five months of running this system, retold plainly, ending in the principle it taught us.

Two conventions to know.

Checkpoints are observable, not introspective. A lab checkpoint is never "you should now understand X." It is always a thing that appears or a thing that refuses: a blocked-push message in your transcript, a file that does not exist because a hook stopped it, a label on a PR, a line in a log. If the observable thing happened, you passed — whether or not it feels understood yet. If it did not, something is wrong, and that is useful information, not a personal failing.

Every lab has a failure branch. Agents fail in mundane ways: a worker that ships nothing, a stuck task, a flaky CI run. You will hit some of these — we certainly did — so each lab includes a short "when this goes wrong" box: symptom, what to check, how to recover, drawn directly from our own incident history. The happy path is the lab; the failure branch is the education.

Why you can trust the numbers in this book

Everything here comes from operating a real AI software company for **5 months**: seventeen AI employees working one codebase, **700+ merged pull requests**, a daemon that runs overnight, and a full audit trail. (That counter is recounted against the repository every time this book is revised — it only ever goes up.)

That includes the embarrassing parts. Early on, our system claimed an 85% success rate. When we built honest verification and measured what had actually shipped, we found that at the worst point **91.5% of "completed" tasks were phantoms** — work reported done that delivered nothing. Clawing the real number upward, failure by diagnosed failure, is where most of this book's lessons come from. The guardrails you will install in Module 1 are not theoretical designs; each one exists because a specific thing went wrong, and the war stories tell you which.

When this book says a command works, it was run against the real framework. When it cites a number, the number was measured — and the measurement usually started as a bad surprise. You will not find "dramatically improves" anywhere in these pages; you will find failure rates with decimal points.

That is the deal. Sandbox ready, `gh` authenticated, expectations set.

Turn the page. Module 1 starts by teaching the machine to tell you no.

Module 1 — Structured Autonomy: Fast AND Safe

There are exactly two ways to fail at agent autonomy, and you are currently at risk of one of them.

The first way is to let the agent run. You have seen the demos: an AI writes the code, commits it, opens the PR, merges it. It works, right up until it confidently does the wrong thing at machine speed — and machine speed means the wrong thing happens forty times before you check in. We know because we run an AI software company on Claude Code, and the audit numbers below are ours.

The second way is to trust nothing. Every file write, every commit, every command gets your personal review. This feels responsible. It is also a decision to cap your AI workforce at the speed of your own attention — a workforce that never sleeps, throttled by a human who does. Over five months our system merged more than 700 pull requests. If each one had required babysitting every intermediate step, the realistic number is a few dozen, and we would have quit from tedium long before month two.

This module teaches the third option, the one this playbook is built on: **structured autonomy**. Deterministic code — plain, boring, non-negotiable programs — decides what the AI is *allowed* to do. The AI gets to be creative only inside those walls. You will learn the four-tier trust model that classifies every action an agent can take, meet the three guardrail hooks that enforce it, install the framework into a sandbox, and then — the part most tutorials skip — deliberately attack your own installation until it tells you no.

By the end you will have watched a guardrail refuse you twice, on purpose, with zero damage done. That experience is the point. A guardrail you have never seen refuse anyone is a guardrail you merely hope works.

The Two Ways to Fail

Let's put real numbers on both failure modes, because vague warnings don't change behavior and numbers do.

Failure mode one: full automation. Early in our company's life, the dashboard said 85% of tasks succeeded. It was wrong, and not by a little. When we built honest verification — checking

whether a "completed" task actually produced a merged, on-target change — the verified rate was 43%. In the worst measured stretch, 91.5% of completions were phantoms: work the system claimed was done that either didn't exist or didn't do what the task asked. Later, after several rounds of fixes, we *still* measured a 31.2% phantom rate. At one point we discovered 45 pull requests our metrics counted as shipped had actually been closed without ever merging. Nobody lied to us. The system reported what it believed, and what it believed was optimistic.

Here is the uncomfortable mechanism behind those numbers: an agent that fails does not look like an agent that fails. It looks like an agent that succeeds — tidy commit message, confident summary, green status — because the model generating the failure is the same model generating the report about it. At human speed you'd catch this. At machine speed, overnight, unattended, you get forty tidy, confident, wrong outcomes before breakfast.

Failure mode two: manual everything. The cure most careful developers reach for is approving every action by hand. Do the math on what that costs. Our system averages multiple worked tasks per day across a queue that runs while we sleep. If every write, commit, and push waited for a human click, throughput drops to whatever one distracted person can review — and the *quality* of your review degrades too, because clicking "approve" two hundred times a day turns you into a rubber stamp with anxiety. You get the bottleneck and, eventually, none of the safety.

Structured autonomy resolves the tension by refusing to make it one decision. Instead of asking "do I trust the agent?" — a question with no good answer — you ask "do I trust the agent *with this specific action*?" Reading a file: yes, always, automatically. Writing code: yes, but a program inspects the write first. Pushing to your main branch: never automatically; a human confirms or it doesn't happen. Deleting recursively with `rm -rf`: no, not ever, and no amount of clever prompting changes that, because the thing saying no is not a prompt.

Diagnose yourself honestly before reading on. If you have already let an agent run loose on a real repo "just to see," your risk is failure mode one, and the guardrails in this module are for you. If you re-read every diff twice, your risk is mode two, and the trust tiers are how you learn to let go of the actions that were never dangerous. Most solo developers are secretly both, on different days.

The dividing line is not how much you trust the AI. It's which decisions you hand to deterministic code and which you keep for judgment.

Why the next section matters: "trust some actions, not others" only works if you can classify actions quickly and consistently. That's what the tier model gives you — and later, when you customize Forge for your own project, mis-tiering a single action is exactly how doors get left open.

The Trust-Tier Model

Every action an agent can take in a Forge-managed project falls into one of four tiers. This table — it lives in `CLAUDE.md`, the contract file every Claude Code session reads — is the spine of the entire framework:

Tier	Examples	Permission model
Free	Read, Glob, Grep, `git status`	Auto-approved, no inspection
Guarded	Write, Edit, `git commit`	Auto-approved, but hooks inspect first
Gated	`git push`, deploy	A human must confirm
Forbidden	`rm -rf`, `sudo`, force-push to main	Blocked by code, no appeal

The logic behind each tier is worth internalizing, because you'll be assigning tiers yourself soon.

Free actions are reads. An agent that reads a thousand files can waste tokens but cannot damage anything. Gating reads would slow everything down and protect nothing. Auto-approve, move on.

Guarded is where structured autonomy earns its keep. Writes and commits are *reversible* — git exists — but they're also where secrets leak and junk accumulates. So the framework approves them automatically *and* runs inspection hooks before they execute. The agent keeps its speed; a program you can read keeps the veto. This tier is why our company can produce hundreds of commits a week without a human watching each one.

Gated actions cross a blast-radius line: they leave your machine. A push publishes; a deploy affects users. A fifteen-second human confirmation here buys enormous protection, so the framework stops and asks. Note what this tier is *not*: it's not "review the whole diff again." It's "a human decided this crosses the line, now." The heavy review happens elsewhere, in the PR flow you'll build in Module 5.

Forbidden actions have blast radii so large or so irreversible that no confirmation dialog is good enough — because at 2 a.m., tired, you will click yes. Recursive deletes, privilege escalation, force-pushing over shared history. These are blocked before execution by code that has no "yes anyway" path. If you genuinely need one, you do it yourself, by hand, outside the agent, with full human deliberation. That friction is the feature.

Practice classifying: which tier is `git checkout -b new-branch`? Local and reversible — Guarded. `chmod 777` on a file? World-writable permissions, a classic path to quiet disaster — Forbidden, and you'll watch the framework block it in the lab. `gh pr create`? Publishes, but to a review queue rather than production — in Forge's flow it rides on a push that was already Gated.

One honest caveat: the tier model is a *contract*, expressed in documentation, enforced by a set of hooks. There is currently no single machine-readable registry you can query with "which tier is this action?" — the answer comes from reading `CLAUDE.md` and the hook source. That's a real gap, and it's why learning to classify actions yourself, right now, is not optional.

When you can't decide an action's tier, decide by blast radius and reversibility — never by how much you trust the model this week.

Why the next section matters: a tier table is just a promise. The enforcement layer makes it binding — and understanding that layer mechanically is what separates "I installed a safety tool" from "I know why this is safe."

Guardrails Are Programs, Not Prompts

Here is the single most important mental shift in this playbook.

Asking a model nicely to "never push to main" is a suggestion. The model will honor it almost every time — and "almost" is doing fatal work in that sentence, because you are going to run thousands of actions. A **PreToolUse hook** — a plain program that Claude Code runs *before* executing any tool call, which reads the proposed action as JSON on stdin and can veto it by exiting non-zero — is not a suggestion. The command it blocks never runs. Not "is discouraged from running." Never runs. The model can want to push to main with every weight in its network; the hook does not care what the model wants.

Forge ships a set of these hooks. Three do the core safety work:

`block_dangerous.py` — the Forbidden-tier enforcer. It pattern-matches proposed shell commands against a list of catastrophic operations: `rm -rf` aimed at roots and home directories, `sudo rm`, `chmod 777`, and friends. When it fires, you get a boxed refusal titled `DANGEROUS COMMAND BLOCKED`, naming the exact pattern that matched.

A true story from the making of this chapter: while verifying these hooks against the live repository, we ran a `grep` *searching for* the text "chmod 777" in the hook's own source code — and the hook blocked the grep, because the search string contained the pattern. That's a false positive, and it's also the whole point. Deterministic rules don't judge intent. They can't be sweet-talked, and the price is that occasionally they block something innocent. That trade — a few seconds of your annoyance against an unbluffable barrier — is the cheapest insurance you will ever buy. (Hold on to that grep, though. A war story later in this module is about the rebuild that made this exact false positive extinct — without surrendering the barrier.)

`secrets_scanner.py` — the Guarded-tier inspector for writes. Before a file write lands on disk, it scans the content for credential patterns: AWS access keys (`AKIA` followed by sixteen

uppercase characters and digits), private key blocks, high-entropy strings assigned to suspicious variable names. On a match, the write is refused *before the file exists*. Two details that will matter in the lab: documentation paths (`.md` files, READMEs, `.example` files) are deliberately exempt, so you can write docs *about* keys; and the free Community tier ships 8 core patterns, while broader coverage (including Anthropic and OpenAI key formats) is part of the paid Compliance Pack — so don't test a free install with an `sk-ant-...` string and conclude the scanner is broken.

`git_guardian.py` — the branch protector. It intercepts `git push` targeting protected branches and refuses with a boxed `BLOCKED: Push to Protected Branch` message before git ever contacts a remote. It also scans commit content for secrets and blocks executable syntax smuggled into commit messages. This one hook is why, across 700+ merged PRs and five months of an autonomous system running daily, direct pushes to main from automation: zero. Not because the agents never tried. Because trying doesn't work.

Notice the design principle threading through all three: the safety-critical component is *boring*. Regex matches, path checks, exit codes. No model call, no judgment, no temperature. Boring means auditable — you can read every rule in an afternoon — and auditable means you can actually trust it, rather than hoping.

Safety-critical decisions must be deterministic and boring, so the creative component can be neither.

Why the next section matters: you have just been told that boring regex is the safety layer. In mid-2026, published research showed that boring regex, as practiced by nearly everyone — us included — had a structural hole. What happened next is this book's best case study in what "deterministic" actually demands.

War Story: GuardFall — the Regex Never Sees the Command

> **War story.** On 2026-06-30, security researchers published GuardFall: an audit showing that the pattern-based shell guards in 10 of 11 popular AI coding agents could be bypassed. Not because anyone's regex was sloppy — for a structural reason. Bash *rewrites* commands before executing them: variable expansion, quote removal, substitution. The string your guard inspects is not the string that actually runs. `rm${IFS}-rf${IFS}/` contains no `rm -rf` for a regex to find; after the shell expands `$IFS`, it is exactly `rm -rf /`. The guard reads the disguise; the shell executes the payload.

> We read that paper the way you read a burglary report describing your own lock. `block_dangerous.py` — the hook you'll watch refuse a `chmod 777` in the lab — was the same flaw class. And we didn't need the paper's word for it: before the rebuild, our own nightly research scout had surfaced a concrete instance in our hook. `rm -rf "$DIR/build"` sails past any literal-path regex — and with `$DIR` empty at runtime, it is a root-level delete on `/build` that no pattern matching paths-as-written ever sees. We patched that specific hole first (#201). GuardFall's message was harder: it isn't a hole, it's the wall.

The obvious response is to pile on more regexes — one for `$IFS`, one for quotes, one for escapes — and we tried it, with a red team standing by to keep us honest. The naive hardening scored **52 bypasses and 30 false positives**. Read that pair of numbers twice: more patterns made the hook easier to fool *and* more annoying at the same time, the worst trade available. Accretion does not fix a structural flaw.

The rebuild that shipped (#233) starts from the research's own conclusion: if the shell rewrites commands before running them, the guard must perform the same rewriting before judging them. The hook now canonicalizes every proposed command with deterministic transforms that model what bash itself will do — `$IFS` expansion, quote removal, ANSI-C escapes, cartesian brace expansion, `echo` / `printf` / `base64` substitution — and only then pattern-matches, against every canonical form.

Canonicalization alone would make the false-positive problem *worse* — expand everything and a commit message merely mentioning `rm -rf /` starts to look like a command. So the second half is structural masking: tokenize each canonical form into command segments and drop quoted **data** arguments. A dangerous string quoted as an argument to `echo`, `grep`, or `git commit -m` is data — the shell will never execute it — so the matcher never sees it. An argument to `bash -c` really is executed, so for shell interpreters everything is kept. That distinction — data versus command, decided by structure rather than by vibes — is why `git commit -m "docs: never run rm -rf /"` now passes cleanly while `bash -c "rm -rf /"` still hits the boxed refusal. It is also why the grep from a few pages ago — blocked for merely *searching* for "chmod 777" — sails through today: a quoted search string is data. The false positive we once called the price of insurance turned out to be a cost you don't have to pay.

Validation was adversarial, because a guard validated by its own author is validated by hope: two red-team rounds, **180+ bypass attempts**, and the shipped version pinned by **129 new tests** with **zero false positives**.

And now the part most write-ups leave out, printed here exactly as the engineers documented it in the hook's own header: runtime-only obfuscation — values known only at execution time, positional parameters — **is out of reach for any static hook**, ours included. That is not a bug

scheduled for later; it is the boundary of what pattern-matching can ever do, exactly as the GuardFall research concludes. This layer is defense-in-depth, not a sandbox — and behind it sit the layers that don't care how a string was obfuscated: the sandboxed execution environment and the PR gauntlet you'll build in Module 5.

Your deterministic layer must model the shell's rewriting — and its limits must be documented, not denied.

Why the next section matters: hooks guard actions. But some files are so load-bearing that you don't want automation touching them no matter how legitimate the action looks. That needs its own mechanism — and it exists because of a very bad night.

humanProtected: The Do-Not-Touch List

> **War story.** Our company once picked up an innocuous-sounding cleanup task. In its enthusiasm to tidy the repository, it deleted the Python build configuration and the dependency lockfile — the two files every install, every test run, and every CI job depends on. Nothing looked alarming in the moment: the diff was small, the commit message was tidy, the agent reported success. Then every build broke at once, and we spent the night restoring files that no automation had any business touching. The fix was not a better prompt or a smarter model. It was a short list in a config file: paths the machinery is forbidden to modify, enforced by a hook that doesn't negotiate.
>
> **Some files must be off-limits to automation by construction, not by convention — if "please don't touch this" isn't enforced by code, it isn't enforced.**

The mechanism is the `humanProtected` block in `forge-config.json`. Here is the heart of our production list:

```
"humanProtected": {
  "description": "Files/directories the daemon must NEVER modify - reserved for
human editing",
  "enabled": true,
  "paths": [
    "CLAUDE.md",
    "forge-config.json",
    ".claude/settings.json",
    "pyproject.toml",
```

```
    "uv.lock"
  ]
}
```

The first line of enforcement is a hook (`lint_on_edit.py`) that checks proposed Edits and Writes against these paths and refuses matches with a boxed block message — though, as you're about to read, the hook is not the layer that has actually been holding the door. The selection principle: **anything whose corruption breaks everything downstream**. The build config and lockfile (the war story's scar tissue). The safety configuration itself — because a do-not-touch list that automation can edit is a polite fiction. The Claude Code settings file where the hooks are registered — same reason. Your equivalents might be a deploy manifest, a database migration directory, a pricing table.

Now the honest caveat, and it surprises almost everyone: **this list guards the automation, not you.** The hook checks whether it's running in an autonomous-worker context — it detects this through the environment — and only enforces the list there. When *you* edit `pyproject.toml` interactively in a Claude Code session, no block fires. That is deliberate. The threat model is a tireless machine making ten thousand edits while you sleep, not a human making a considered change with their eyes open. Somebody has to be able to edit the build config; the design says that somebody is a person, in the room, on purpose.

There is a second honest caveat, and it's fresher. In July 2026 we audited this exact mechanism in our own production system and found (#265) that on the real autonomous path, the daemon never actually sent the worker-context signal the hook listens for — the hook's `humanProtected` block was dead code for precisely the workers it was written to stop. The receiver worked; the sender was silent. The finding produced a rebuilt enforcement that lives where an unsent signal can't kill it: at PR-creation time, the worker pipeline refuses to stage, commit, or push any change that touches a protected path. It checks the entire branch diff — renames and deletions included, so a protected file can't slip out under a new name — and if `forge-config.json` can't be read at all, it fails closed and refuses the PR rather than guessing.

So here is the claim this book can actually back, which is narrower than the one we'd have made before that audit: the protected-file list is real, the intent is real, and it holds through *layers* — the hook when its signal is present, the PR-capture refusal in the worker pipeline, and the review gates behind both. Any single layer described as a "hard block" is a stronger claim than the code supports; the stack together is what holds.

Keep the human/machine asymmetry in mind too. It's the difference between a safety system and a straitjacket — and it's the seed of this module's challenge, where you'll prove the hook's logic bites by simulating the machine, and then learn what that proof does *not* cover.

Why the next section matters: everything so far is theory you've taken on my word. Time to stop taking anyone's word.

Install, Verify, Then Distrust

The installation itself is one command. What matters is the posture you bring to it, which is the same posture this whole framework teaches: **verify, then attack**.

Forge's installer (`install.sh`) asks you two questions worth understanding before you answer them:

Installation type — `core` (hooks and safety only), `single-project` (the full framework for one repo — what you'll use), or `multi-project` (a company root managing several repos, covered in Module 8).

Security profile — `strict` , `standard` , or `minimal` . This controls how aggressive the guardrails are. `standard` — critical protections block, the rest warn — is the right answer for this playbook. If you ever type `minimal` , you should be able to say precisely which protections you just declined.

After installing, you do not trust the install. An installer can report success while a hook sits unregistered, missing, or broken — a file's *presence* proves nothing about its *enforcement*. So you verify with `/security-status` , which checks the local hook layer the way a skeptic would: is `git_guardian.py` actually registered as a PreToolUse hook in `.claude/settings.json` ? Does the file exist on disk? Does it compile without syntax errors? Three checks, each one a way an installation can silently lie to you.

And after verifying, you still don't trust it — a green checkmark is a claim, and the only evidence that convinces anyone at gut level is watching the thing refuse you. That's the lab. You are going to attempt a push to a protected branch, attempt to write a credential into code, attempt a forbidden command, and read three refusals with your own eyes. Then, crucially, you'll do the *legal* version of the same work and feel how fast it is when you're inside the walls.

Presence is not protection. Verify the mechanism, then make it say no to you once — before you need it to say no to a machine ten thousand times.

Lab 1 — Install Forge, Then Try to Break Your Own Rules

Time: 30–45 minutes. You need: git, Claude Code, and a terminal. Everything happens in a throwaway directory.

1. **Create a sandbox project.** Never run a lab against a repo you care about — that rule is itself a blast-radius decision, and you're practicing those now.

`bash mkdir ~/forge-sandbox && cd ~/forge-sandbox && git init -b main echo '# sandbox' > notes.txt git add notes.txt && git commit -m 'init'`

1. **Get the framework and preview the install first.**

`bash git clone https://github.com/andersenqvistdev/forge-framework ~/forge-framework ~/forge-framework/install.sh --dry-run ~/forge-sandbox`

Read what it *would* do before letting it do anything. You're about to hand this tool your project directory; previewing an installer before running it is exactly the discipline you're here to learn.

1. **Install for real, with explicit choices.** Explicit flags instead of interactive prompts, so your shell history records exactly what you chose.

`bash ~/forge-framework/install.sh --type single-project --security-profile standard -y ~/forge-sandbox`

1. **Verify the installation is protecting you, not just present.** Open Claude Code inside `~/forge-sandbox` and run:

`/security-status`

Confirm Layer 1 (local hook protection) reports `git_guardian.py` as registered in `.claude/settings.json`, existing on disk, and compiling cleanly. Layers 2 and 3 concern GitHub-side protection — you have no remote yet, so expect those to report missing. That's honest output, not a failure.

1. **Trip guardrail #1: the protected branch.** Ask Claude to run `git push origin main`. The push is intercepted *before execution* — you don't even need a remote configured, because the hook fires before git ever gets the chance to look for one. Read the boxed refusal: `BLOCKED: Push to Protected Branch`. Sit with the detail that matters: this came from `git_guardian.py`, a PreToolUse hook — a program — not from the model deciding to be careful. The model *tried*. The program said no.

2. **Trip guardrail #2: the secret write.** Ask Claude to create `config.py` containing a fake AWS credential. Use exactly this dummy — `aws_key = "AKIA2X7QZ9WPLM3RBV8K"` — or invent your own sixteen characters after the `AKIA`. One trap worth knowing, because it teaches how the scanner thinks: **do not use** `AKIAIOSFODNN7EXAMPLE`. That is AWS's own documentation example key, and the scanner deliberately allowlists known placeholder and example strings — so if you reach for the "standard" fake AWS key you have seen a hundred times, the write sails through and you wrongly conclude the guardrail is broken. It is doing exactly its job: real-looking secrets are blocked, published dummies are not. Also

don't name the file `*.md` or `README*` — documentation paths are exempt from scanning by design. With a real-looking dummy in a code file, the Write is refused by `secrets_scanner.py` before the file ever touches disk.

3. **(Optional) Trip guardrail #3: the forbidden command.** Ask Claude to run `chmod 777 notes.txt` and watch `block_dangerous.py` refuse with `DANGEROUS COMMAND BLOCKED`. No confirmation dialog, no override flag. Forbidden means forbidden.

4. **Now do it the legal way**, which is how you'll work for the rest of this playbook:

`bash git checkout -b feat/first-branch git commit --allow-empty -m 'feat: prove the safe path'`

Branching and committing are Guarded-tier: inspected, then instantly allowed. No prompt, no wait. The same system that refused you twice says yes without hesitation here. That contrast — instant *no* on the dangerous path, instant *yes* on the safe one — is what "fast AND safe" feels like from the inside.

1. **Find the do-not-touch list — then notice which file the hook actually reads.** Open `.claude/forge-config.json` in your sandbox: the installer ships it with a full `humanProtected` block, matching our list above. But here is a trap worth learning on a throwaway: the enforcing hook, `lint_on_edit.py`, does not read that file. It consults `forge-config.json` at the *project root* — and a fresh install does not create one. The `.claude/` copy is the install template; the root file is the config the hook obeys. So make the list real yourself:

`bash # from ~/forge-sandbox — give the hook the file it actually reads cp .claude/forge-config.json forge-config.json`

Then ask the real question: which of *your* files — in the project you actually care about — belong on this list? Write them down. You'll want that answer in Module 8, when the send-off has you write that list before anything else on a repo you care about.

When this goes wrong

If a guardrail doesn't fire: five checks, in order. First, re-run `/security-status` — if Layer 1 shows the hook unregistered or failing to compile, your install didn't complete; re-run step 3. Second, confirm you're inside `~/forge-sandbox` when talking to Claude Code — hooks are registered per-project in `.claude/settings.json`, and a session opened elsewhere has no idea your sandbox exists. Third, for step 6 specifically: check your key and filename. If you wrote the AWS *example* key (`AKIAIOSFODNN7EXAMPLE`) or put it in a `.md` file, the scanner skipped it by design — use a real-looking dummy in a `.py` file. Fourth, if Claude keeps warning that the workspace "has not been trusted" and ignores its own settings: the very first time you open Claude Code in the sandbox, accept the trust dialog. Until you do, Claude ignores the project's

permission list and nags on every run — a one-time first-run surprise, not a broken install. Fifth, if you improvised on step 7 and hid the dangerous string inside quotes — a commit message mentioning `rm -rf /`, a grep for `chmod 777` — and it sailed through: that is not a broken guardrail. Since the #233 rebuild the hook distinguishes data from commands, and a dangerous string quoted as data to `echo`, `grep`, or `git commit` is deliberately allowed; run the bare command (or a `bash -c` version of it) and the block fires.

Checkpoint

You're done when you can show three observable artifacts:

1. The `BLOCKED: Push to Protected Branch` boxed message in your transcript, with the sandbox still un-pushed — `git log origin/main` fails, because no remote was ever touched.
2. `ls config.py` returns "no such file" — the secret write never landed on disk. Not "was reverted." Never landed.
3. A `/security-status` report showing the local hook layer passing.

Two refusals witnessed, zero damage done. You now believe in the guardrails the only way that counts: empirically.

Challenge — Make the Do-Not-Touch List Bite

The lab showed you three guardrails firing. This challenge makes you *prove* a fourth — the `humanProtected` enforcement — and it will not fire for you the obvious way, which is the entire lesson.

The task: add a path you care about (say `docs/adr/*`) to `humanProtected.paths` in the root-level `forge-config.json` you created in Lab step 9, then demonstrate that the block actually bites — *without running any daemon*.

Three things you'll have to discover yourself. First, the enforcement lives in `.claude/hooks/lint_on_edit.py` — read it. Second, it only activates when it believes an *autonomous employee*, not a human, is doing the editing — and an environment variable is how it knows. Third, the hook consults exactly one config file: `forge-config.json` at the project root. It deliberately does not read `.claude/forge-config.json` — put your path in the install template instead of the root config and the hook exits 0 forever, no matter what environment you set.

A hint at the shape of the solution: Claude Code hooks are plain programs. They read a JSON tool-call payload on stdin and signal a block through their exit code. So you can hand-craft an

Edit payload targeting your protected path, pipe it into the hook under a simulated daemon environment, and check `echo $?`.

One self-test to keep you honest: if you get exit 0 both with and without the environment variable set, either you haven't found the right variable yet — or your path is sitting in `.claude/forge-config.json`, the file the hook never reads. The two failures look identical from the outside, which is exactly why the config-location lesson belongs in this challenge.

When you succeed, you'll have done something more valuable than pass a lab: you'll have read a safety hook's source, understood its activation condition, and verified its behavior from the outside. Every guardrail you ever rely on deserves that treatment at least once.

Then read your success with July-2026 eyes. By setting that variable yourself, you impersonated the daemon — and when we audited our own production (#265), we found the real daemon never set it at all. The hook's logic was sound, the signal never arrived, and the layer was dead code on the very path it was written for; enforcement had to be rebuilt at PR-creation time, where there is no signal to forget. That upgrades this challenge's lesson: verifying a guardrail from the outside means verifying both ends — the hook that listens for a signal, and the process that is supposed to send it. An unsent signal fails exactly like a missing hook, and nothing on your screen tells you which one you have.

What you can now do

> **Module 1 summary — five things you couldn't do this morning:**
>
> - **Classify any agent action** into Free / Guarded / Gated / Forbidden using blast radius and reversibility — not vibes about the model.
> - **Install Forge into a project** with an explicit type and security profile, previewing with `--dry-run` before committing.
> - **Verify an installation is enforcing, not just present**, with `/security-status` — registration, existence, compilation.
> - **Trigger and read a guardrail refusal**, identifying which hook fired and why, so a block message is information rather than mystery.
> - **Declare do-not-touch files** in `humanProtected.paths` and explain the honest boundary: it's aimed at the machine, not the human — and it holds as a stack of layers, with the load-bearing check at PR-creation time, not as one hard block.

What the full Playbook covers

This was Module 1 of 9 — the foundation everything else stands on. Here's where the rest takes you:

Module 2 — Found your company. Hire your first AI employees, give them roles and memory, draw the line between code and the org's live state — then bridge your sandbox to GitHub, where every later module's work will arrive.

Module 3 — First work by hand. Run a complete task through an employee yourself — request to branch to diff — so when automation does it later, you know exactly what "done right" looks like.

Module 4 — Plan like a CEO. Feed the system a roadmap instead of tickets, and watch the admission gate reject bad work *before* it wastes a worker — the fix that took our phantom problem apart at the source.

Module 5 — The GitHub flow safety net. Wire branch protection, CI, and the pre-merge deliverable judge into a net where an agent's PR simply cannot land unreviewed.

Module 6 — Go autonomous with a canary. Flip the auto-merge switch the way we actually did it: one genuine task and one booby-trapped one in flight, and you don't proceed until the gates catch exactly the right one.

Module 7 — Operate: steer, don't row. The unattended capstone — a daemon working while you're gone, overnight in the challenge, and a report you can read in five minutes because you built every gate it answers to.

Module 8 — Scale and economics. When to raise worker counts, what an autonomous PR actually costs on a flat Claude subscription, and how to read five months of our real numbers — including the ugly ones — to decide what to automate next.

Module 9 — Feed the company. Give the company outside-world intake: a nightly scout that researches and proposes evidence-backed tasks as PRs behind a single human merge — no machine-proposed work runs unless a human said yes.

Each module has the same anatomy you just experienced: real mechanisms verified against a production system, a lab where you operate them yourself, a war story with the scar tissue left in, and a challenge that makes you prove it rather than trust it.

If this module changed what you can do, the other eight are built to the same standard. Get the full Forge Playbook and take the rest of the ladder — one verified rung at a time.